More Constrained: Apples or Oranges?

On the road to semantic constraint matching

Document number:	P1375R2
Date:	2019-08-03
Project:	ISO/IEC JTC 1/SC 22/WG 21/C++
Audience subgroup:	*None* (informative update)
Revises:	P1375R1

Reply-to: Hubert S.K. Tong <hubert.reinterpretcast@gmail.com>

Changelog

Changes from R1

- Added a “Current Status” section.
- Updated the abstract to clarify the status of this paper.
- The paper is otherwise unchanged from its previous version.

Changes from R0

- Updated acknowledgements and further considerations based on discussions at the 2019 Kona meeting.

Abstract

At the 2017 Toronto meeting, EWG voted for “partial ordering rules using expression identity” in response to P0717R0’s recommendation for semantic—as opposed to textual—matching for concepts. This paper demonstrates that the implementation of said direction is incomplete given the status quo of the Working Draft. It further explores the nature of both expression identity and type identity, and proposes a framework (not actively being pursued) for atomic constraint identity based on conservative semantics using affirmative-identity matching.

A treatment of the related topic of constraints on non-templated functions is given based on previous discussions.

Contents

Structure

This paper begins its exploration with the application of constraints on non-templated functions since a larger amount of recent discussion has taken place on that topic. It proposes a direction on the treatment of non-templated functions somewhat separately before delving further into the broader topic.

Current Status

The issues described in this paper remain; however, the exploration of the proposed treatment of type identity did not seem practical for C++20. An alternative approach, in the form of P1624, can solve the technical issues raised in this paper. The approach in P1624 makes the `transmogrify` example ordered, which might not match the user intent; however, the case could be made for user error.

Newer developments on the EWG mailing list indicate unease with constraints on non-templated functions. It is no longer clear that the approach presented to EWG at the 2018 Jacksonville meeting (and documented by this paper) is the desired solution, and there is a lack of implementation experience with that approach. It is expected that the issues will be handled via the comment resolution process on the ballot of the upcoming Committee Draft.

Constraints and non-templated functions

Various and sundry utterances in the Working Draft establish the ability to apply constraints upon non-templated functions. So the following is possible:

```
template <unsigned> constexpr bool t = true;
template <typename> concept C0 = t<0u>;
template <typename T> concept C1 = C0<T> && t<1u>;

struct TangerineIMacG3;
using Apple = TangerineIMacG3;
using Orange = TangerineIMacG3;

void compose(Apple *, Orange *) requires C0<Apple>;
int compose(Apple *, Orange *) requires C1<Orange>;

int transmogrify() { return compose(nullptr, nullptr); }
```

A question we can ask ourselves is whether or not the two `compose` functions should be ordered with respect to each other.

As humans, we can say that the first function only constrains the type of its first parameter and that the second one constrains only its second. This leads to the evaluation that the constraints are unrelated (they do not constrain related things), which further leads to the conclusion that the function should be unordered.

Another question we can ask is what the Concepts TS would do in this situation (with the addition of `bool` to the concept definitions). The answer is that normalization would produce "`t<0u>`" as the normalized associated constraints of the first function `compose` and "`t<0u>` ∧ `t<1u>`" as that of the second. The latter subsumes the former because of the atomic constraints `t<0u>` are textually (ODR) equivalent to each other.

Has the situation changed with the Working Draft wording? The answer is *no*. The normalization still produces fairly much the same atomic constraints, except that the atomic constraints `t<0u>` are considered identical as a combination of being the same lexical *expression* and having the same (empty) parameter mapping.

So what went wrong here? The empty parameter mapping is easy to point at (and indeed, it is one reason why we have a clear answer out of the current wording). What we are seeing is that constraints which are non-dependent do not actually constrain template parameters (of course, the functions have no template parameters at all). In the absence of template parameters being constrained, we have no basis for understanding how constraints relate to the elements of the constrained declaration and no basis for understanding whether constraints on one declaration share the same intended relationship with its declaration as constraints on another declaration (and are thus meaningful to order by).

Hopefully with the stage set on why constraints on non-templated functions are problematic to order between in terms of semantic constraint matching, we now move into some more history and technical details as to why supporting such ordering is issue-ridden.

From Concepts Issue 23

Concepts Issue 23 notes the addition of “associated constraints” by the Concepts TS to the definition of a function’s signature where associated constraints is only defined for templates (thus leaving the signature an ill-defined concept). The situation has now slightly changed.

The definition of a function’s signature ([defns.signature]) now refers to the trailing *requires-clause* (if any) of the function. It remains the case, however, that “associated constraints” are defined for templates ([temp.constr.decl]) and not for functions. Thus, we would understandably be at a loss when overload resolution, with hands waving vigorously, asks us to prefer a more constrained non-template function ([over.match.best]) using rules that order declarations based on their associated constraints ([temp.constr.order]).

On top of that, it is unclear how trailing *requires-clause*s that evaluate, as constant expressions, to the same value are to be considered not equivalent for the purposes of declaration matching ([over.dcl]). In particular, [over.dcl] may be applying *equivalent* upon the values produced by interpreting the *constraint-logical-or-expression*s of the trailing *requires-clause*s as constant expressions; in which case, the [over.match.best] verbiage serves at most to prefer constrained declarations over unconstrained declarations. Alternatively, [over.dcl] may instead be applying *equivalent* upon the expressions themselves with the expectation that such equivalence is defined. It appears that such an expectation would be unwarranted, because *equivalent*, with respect to expressions ([temp.over.link]), is defined only for expressions involving template parameters.

The uncertainty conferred by the involvement of template parameters is necessary for implementations to distinguish between cases that are not functionally equivalent. We do not require “implementations to use heroic efforts to guarantee that functionally equivalent declarations will be treated as distinct” ([temp.over.link]). In the absence of template parameters, all well-formed *constraint-logical-or-expression*s that evaluate to `true` *are* functionally equivalent and thus indistinguishable.

We are left with a situation where functions we do not meaningfully distinguish between may participate in an overload set such that we could try to order them based on constraint subsumption. This last promise should probably be a non-goal as the coincidental matching factor is a problem.

Stated more strongly: Differentiating between constraints that are either categorically satisfied or categorically unsatisfied is inconsistent with the treatment of expressions that are functionally

equivalent but not equivalent; the ability to have and to choose between non-templated functions that are identical except in their trailing *requires-clause*s is a lie.

As an aside, [basic.link] does not take trailing *requires-clause*s into consideration; therefore, it prohibits declaring functions with the same name and parameter-type-lists, but with differing trailing *requires-clause*s, to be members of the same namespace in cases like that of a friend declaration. That wording may need to be updated as well.

Previous Discussion Records

The author reported Concepts Issue 23 in June 2015 and a further follow-on in July 2016 that broadly covers the basis of the current write-up. The author suggested, at the time, a proposed direction whereby the signature accounts for the trailing *requires-clause* only by the value and not by the form. These issues were discussed on a CWG teleconference, with the participation of Andrew Sutton, in preparation for the July 2017 Toronto meeting. Coming out of the CWG teleconference, there was an agreement to adopt the aforementioned proposed direction. The procedural clarification given by the CWG chair was that some of the resolutions would be expected to come as part of applying Concepts to the main C++ working draft, with the rest being taken as CWG issues.

The follow-up from Core to EWG occurred at the 2018 Jacksonville meeting with Richard Smith and the author explaining the issue. The committee's motivation for allowing the formation of the overload sets in question was sought and found to be unknown in terms of design intent during that discussion, and parties interested in proposing a solution that supports such overloading were invited to prepare papers.

The papers giving motivation for maintaining overload resolution between non-templated functions on the basis of subsumption relationships has not materialized as of the November 2018 meeting of WG 21. This paper documents wording issues with the status quo, and the author believes that the situation warrants resolution as a CWG issue with some amount of EWG guidance.

Proposed Direction

The following proposed direction is an adaptation of the direction from the CWG teleconference of June 2017.

For functions that are not templated entities, the *constraint-logical-or-expression* in the trailing *requires-clause* (if any) is evaluated as a constant expression in the context of the function declaration. For such functions, the requirement for equivalent trailing *requires-clause*s in [over.dcl] is instead for the result value of the evaluation to be equal between that of one declaration and another. If the evaluation results in `false`, then the declaration is not a definition (thus well-formed programs with multiple *function-definition*s of such a function is possible); the bodies are taken as discarded statements in the style of [stmt.if]. It is ill-formed to befriend such a function.

A function that is not a templated entity without an explicit *requires-clause* is considered to be implicitly `requires true`. It is therefore not possible to provide multiple viable candidates that are identical except for the trailing *requires-clause*.

Discussion of Separable Riders

The above proposed direction contains the following separable riders:

- The prohibition upon befriending "`requires false`" non-templated functions.

- The implicit "`requires true`" property of "unconstrained" non-templated functions.

The prohibition upon befriending stems from the lack of specificity that such a friend declaration would have. While it is possible to implement, not having the proposed restriction is the option that is less compatible with having a differentiation mechanism.

The implicit "`requires true`" comes from the view that explicitly having "`requires true`" does not really make a declaration any more constrained than an "unconstrained" declaration. Having this restriction is *less* compatible with having a future differentiation mechanism than not having this restriction. It is also the case that it may be meaningful to allow an "undecorated" default implementation alongside a set that is constrained upon mutually-exclusive requirements.

On atomic constraint identity

Atomic constraint identity is established by [temp.constr.atomic] in the Working Draft using the lexical origination of the expression and the equivalence of the parameter mappings. We immediately hit the issue that the equivalence is to be determined by the [temp.over.link] rules for expressions, which leaves some question as to how type identity is handled and also gets us straight back into an issue with a need to distinguish between expressions that are functionally equivalent but not equivalent.

This use of *equivalent* in the [temp.over.link] sense for determining atomic constraint identity (albeit now in the parameter mapping component) flies in the face of the direction established in Toronto. The author thus proposes that expressions in the parameter mapping are to be considered equivalent using the same rules (recursively, should they also involve template parameters in their lexical context) as those for atomic constraint identity.

We are now left with a question over the nature of type identity.

We note that requiring a symbolic relationship between types used in a parameter mapping for the purposes of constraint matching appears to be implemented in Saar Raz's clang-concepts (available at https://github.com/saarraz/clang-concepts).

For example, given:

```
template <typename, unsigned> constexpr bool t = true;
template <typename T> concept C0 = t<T, 0u>;
template <typename T> concept C1 = C0<T> && t<T, 1u>;
template <typename> concept C2 = C0<short>;
template <typename> concept C3 = C1<short>;
template <typename> struct A;
template <C2 T> struct A<T> { };
template <C3 T> struct A<T> { };
A<int> a;
```

The partial specializations are considered ambiguous when declaring `a`. There is no symbolic relationship between the respective instances of `short` in the definitions of `C2` and `C3`.

Can we say that these instances of `short` are plainly the same? We may have difficultly finding an implementation strategy that allows us to say so and yet claim that `std::initializer_list<std::size_t>::value_type` is distinct from `std::initializer_list<std::size_t>::size_type`.

The author boldly proposes that using the expression treatment for types is the right answer: types are the same when they emanate from the same location in the source and receive recursively-the-same targets for the template parameters from their lexical contexts.

Given that we are ordering disparate candidates, how do we relate dependent types as emanating from the same source? This is the question at the heart of Concepts Issue 24.

Consider:

```
template <typename> constexpr bool t = true;
template <typename T> concept P = t<T>;
constexpr bool Q = true;

template <typename OrangeType = int> requires P<OrangeType>
void foo(int = 0, OrangeType = 0);

template <typename AppleType = int> requires P<AppleType> && Q
int foo(AppleType = 0, int = 0);

int bar() { return foo(); }
```

On the call to `foo()`, neither `OrangeType` nor `AppleType` participate in the partial ordering aside from the determination of the more constrained template. Their involvement in said determination appears wrong.

Furthermore, both the GCC implementation of the Concepts TS and the clang-concepts implementation believes that `OrangeType` corresponds to `AppleType`. This also seems wrong too, because `OrangeType` is used for the second function parameter of its respective template, and `AppleType` for the first. Again, we encounter coincidence—of the positional form this time—in the place of principled matching.

The author proposed, at the time, and is proposing now, that the determination of the more constrained template should use only atomic constraints that relate to template parameters that participate in the usual partial ordering; and, furthermore, that for the purposes of establishing the identity of the dependent types in the parameter mappings, the template argument deduction performed for partial ordering (which succeeded in both directions) be used to establish any direct correspondence between template parameters from different candidates. Note that it naturally follows from this that non-dependent atomic constraints do not participate in partial ordering.

In summary, atomic constraint identity should require that the atomic constraint applies to parameters that correspond to each other for the purposes of partial specialization. Furthermore, following from P0717, the parameters of a constraint should be considered identical only if they emanate from the same source. This is, atomic constraints are identical if and only if they are formed from the same expression, the targets of the parameter mappings originate from the same *expression* or *type-id* and each, in turn, have the same parameter mappings; or the targets are template parameters directly-related in the course of partial ordering. Finally, atomic constraints that do not directly or indirectly involve a parameter mapping to a target of the last kind are ignored for the purposes of partial ordering.

Further considerations

It is acknowledged that usage of type transformations such as `remove_reference_t` in multiple candidates of an overload set or set of specializations would lead to non-matching where matching behaviour was intended. Further exploration is needed. In particular, discussions at the 2019 Kona

meeting indicate that alternative solutions may be possible by insisting on a more accurate match between the candidates before attempting partial ordering based on subsumption. The search for useful examples to guide the direction is ongoing.

Acknowledgements

The author would like to thank—for their feedback on the subject of this paper—S. Davis Herring, Richard Smith, Casey Carter, Andrew Sutton, and any others who have been missed. As usual, any remaining mistakes are the responsibility of the author.